UNIT V
PL-SQL

Why PL/SQL?

Though SQL is a programming language, it has some disadvantages :

1. Procedural capabilities :SQL does not have any procedural capabilities i.e. looping, branching that is must for data testing before its permanent storage.
2. Speed of data processing : SQL statements are passed to the Oracle Engine one at a time. This adds to the traffic on the network, thereby decreasing the speed of data processing, especially in multi-user environment.
3. Error Handling: While processing an SQL sentence, if an error occurs, then the oracle engine displays its own error messages. SQL has no facility for handling errors .

What is PL/SQL?

As the name suggests, PL/SQL is a **superset** of SQL.

PL/SQL is a block-structured language that enables developers to combine the power of SQL with procedural statements.

PL/SQL bridges the gap between database technology and procedural programming languages.

Advantages of PL/SQL

1. PL/SQL provides facilities of conditional checking, looping and branching.
2. PL/SQL sends an entire block of SQL statements to the oracle engine all in one go, so it helps to reduce network traffic
3. PL/SQL provides error handling facility.
4. It allows declaration and use of variables in the blocks of code.
5. Via PL/SQL, all sorts of calculations can be done quickly without the use of Oracle engine.
6. Applications written in PL/SQL are portable to any computer hardware and operating system. Hence PL/SQL code blocks written for a DOS version of oracle will run on its Linux/Unix version.

PL/SQL Block Structure

Syntax of PL/SQL Block Structure:

```
DECLARE        --optional
               <declarations>
               .
               .
               .
BEGIN          --mandatory
               <executable statements. At least one executable statement is mandatory
               .
               .
               .
EXCEPTION      --optional
               <exception handler>
               .
               .
               .
END;           --mandatory
/
```

Sections Of PL/SQL Block

- Declare Section – Code block starts with a declaration section in which memory variables and other Oracle objects can be declared and if required initialized. Once declared they can be used in SQL statements for data manipulation
- Begin Section – Consists of set of SQL and PL/SQL statements which describe processes that have to be applied to table data. Actual data manipulation, retrieval, looping and branching constructs are specified in this section
- Exception Section – Deals with handling of data errors that arise during execution of the data manipulation statements, which make up the PL/SQL block code block. Errors can arise due to syntax, logic and or validation rule violation
- End Section – Marks the end of PL/SQL block

PL/SQL Data Types

Scalar – Single values with no internal components

Composite – Data items that have internal components

Reference – Pointers to other data items.

Large Objects(LOB) –Pointers to large objects such as text, graphic images, video clips

PL/SQL Data Types

The default data types declared in PL/SQL are number, char, varchar, date and Boolean

Raw types are used to store binary data of fixed length and its maximum length is 32767 bytes

Long Raw is similar to Raw but its maximum length is 2GB

ROWID data type is the same as the database ROWID pseudo-column type which holds a rowid and can be considered as a unique key for every row in the database.

BLOB (Binary LOB) – Stores unstructured binary data up to 4GB in length. A BLOB could contain video or picture information

CLOB (Character LOB) – Stores single byte character up to 4GB in length. This might be used to store documents

BFILE (Binary File) – This stores a pointer to read only binary data stored as an external file outside the database

1.Scalar Data Types

Scalar Data Types divided into 4 groups:

- Numeric Types: int, integer, number, float
- Character Types:
- char, character, varchar, varchar2, string, raw,
- rawid
- Boolean Types : boolean
- Date Types: date

Declaring Variables

```
a int;
```

```
sum number(4);
```

```
In_stock boolean;
```

Assigning Values to a variable

Three ways to assign values to a variable

- Using Assignment Operator(:=)
- Eg. a int := 10;
- bonus := empsal*0.10;
- 2. The second way to assign values to a variable is by selecting database values into it.
- select empsal * 0.10 into bonus from emp where eid=2;
- 3. Using IN and OUT Parameter

Basic Examples

Display values of variables using PLSQL block.

```
--
Set Serveroutput on
declare
   i int:=10;
   j int:=20;
begin
 DBMS_OUTPUT.PUT_LINE('value of i=' || (i));
 DBMS_OUTPUT.PUT_LINE('value of j=' || (j));
end;
/
```

Display addition of two numbers using PLSQL block.

```
--
Set Serveroutput on
declare
   i int:= 5;
   j int:=10;
   k int:= i + j;
begin
DBMS_OUTPUT.PUT_LINE('value of k=' || (k));
end;
/
```

Performing Arithmetic Operations accepting value from user

```
--
set serveroutput on
declare
 i int;  j int; k int;
begin
i:=&i;
j:=&j;
 k:=i + j;  DBMS_OUTPUT.PUT_LINE('Addition value of k=' || (k));
 k:= i - j;  DBMS_OUTPUT.PUT_LINE('Subtraction value of k=' || (k));
 k:= i *j;  DBMS_OUTPUT.PUT_LINE('Multiplication value of k=' || (k));
 k := i/j;  DBMS_OUTPUT.PUT_LINE('Division value of k=' || (k));
end;
```

Types of PL/SQL block

Anonymous blocks

Named Blocks

Anonymous blocks:

- Anonymous blocks are PL/SQL blocks which do not have any names assigned to them. They need to be created and used in the same session because they will not be stored in the server as database objects.
- Since they need not store in the database, they need no compilation steps. They are written and executed directly, and compilation and execution happen in a single process.
- Below are few more characteristics of Anonymous blocks.
- These blocks don't have any reference name specified for them.
- These blocks start with the keyword 'DECLARE' or 'BEGIN'.
- Since these blocks do not have any reference name, these cannot be stored for later purpose. They shall be created and executed in the same session.
- They can call the other named blocks, but call to anonymous block is not possible as it is not having any reference.
- It can have nested block in it which can be named or anonymous. It can also be nested in any blocks.
- These blocks can have all three sections of the block, in which execution section is mandatory, the other two sections are optional.

Named blocks

- Named blocks have a specific and unique name for them. They are stored as the database objects in the server. Since they are available as database objects, they can be referred to or used as long as it is present on the server. The compilation process for named blocks happens separately while creating them as a database objects.
- Below are few more characteristics of Named blocks.
- These blocks can be called from other blocks.
- The block structure is same as an anonymous block, except it will never start with the keyword 'DECLARE'. Instead, it will start with the keyword 'CREATE' which instruct the compiler to create it as a database object.
- These blocks can be nested within other blocks. It can also contain nested blocks.
- Named blocks are basically of two types:
- Procedure
- Function

Differences Between Anonymous Blocks And Subprograms

Anonymous Blocks	Subprograms
Unnamed PL/SQL blocks	Named PL/SQL blocks
Compiled every time	Compiled only once
Not stored in the database	Stored in the database
Cannot be invoked by other applications	Named and therefore can be invoked by other applications
Do not return any values	Subprograms called functions must return values
Cannot take parameters	Can take parameters

The Character set

1. Uppercase alphabets {A-Z}
2. Lowercase alphabets {a-z}
3. Numerals {0-9}
4. Symbols

   ---Ordinary symbols: () + - * / < > = ; % ‘ ” []

   ---- Compound symbols: <> != <= >= :=
5. Lexical Units

   ----Words used in a PL/SQL block are called Lexical Units, which can be classified as follows:

1. Delimiters(Simple and compound symbols)
2. Identifiers
3. Literals
4. Comments

1.Delimiters

A Delimiter is a simple or compound symbol that has a special meaning to PL/SQL

Simple symbol consists of only one character

Eg: addition(+) ,subtraction(-) , multiplication(*), division(\) operators

Compound symbol consists of two characters

Eg: Assignment(:=), association(=>), concatenation(||), relational (<>)

2.Identifiers

Identifier consists of a letter optionally followed by more letters, dollar signs, underscores and number signs.

Other characters such as hyphens, slashes and spaces are not allowed.

Eg. n,num,num2,credit$limit, credit_limit --- allowed

Credit&limit, credit-limit, credit limit --- not allowed

3.Literals

1)Numeric Literal (integer or float)

Eg. 25, 6.34

2)String Literal

Eg. “Hello World”

3)Character Literal

Eg. ‘A’ ,‘y’

4) Logical (Boolean) Literal

Eg. True, False or NULL

4.Comment

Single line comment (--)

Multi line Comment (/* */)

PL/SQL Blocks

- Anonymous blocks are unnamed executable PL/SQL blocks which can neither be reused nor stored for later use.
- Procedures and Functions are named PL/SQL blocks which are also called as SUBPROGRAMS These sub programs are stored and compiled in database. The block structure of the subprograms is like that of anonymous blocks
- An anonymous PL/SQL block is a block of PL/SQL that's coded within a script
- To execute a PL/SQL block you must code a front slash (/) after the END keyword
- The structure of anonymous block is

```
DECLARE
        Variables  declarations
BEGIN
        Executable statements
EXCEPTION
        Error handling code
END
/
```

%TYPE Attribute

- The %TYPE attribute provides further integration.
- PL/SQL can use the %TYPE attribute to declare variables based on definitions of columns in a table. Hence if the columns attributes are changes the variables attributes will change as well.
- This provides for data independence, reduces maintenance costs and allows programs to adapt to changes made to the table
- %TYPE declares a variable or constant to have the same data type as that of a previously defined variable or of a column in a table or view
- When referencing a table a user may name the table and the column or name the owner the table and the column
- The syntax for declaring the variable with %TYPE attribute is
  - <variable name> <tablename.fieldname%TYPE>;
  - Eg v_eno Emp.Empno%TYPE

%ROWTYPE Attribute

- The %ROWTYPE attribute is used to declare a record that can hold an entire row of a table or a view
- The fields in the record take their names and data types from the columns of the table or view
- The record can also store an entire row of data fetched from a cursor or cursor variable
- The syntax for declaring a %ROWTYPE variable is <variable_name> <table_name>%ROWTYPE;
- The main advantage of using a %ROWTYPE is that it simplifies maintenance that is it ensures that the data types of the variables declared with this attribute change dynamically when the underlying table is altered
- The %ROWTYPE attribute is particularly useful when you want to retrieve an entire row from a table

Using Sequences in PL/SQL

- Sequences created in database can also be used in PL/SQL block
- After creating the sequence we can use the NEXTVAL and/or CURRVAL attribute of sequence in the BEGIN section of the PL/SQL block by transferring the sequence value into a variable
- The syntax that can be used is

  SELECT <sequence_name>.NEXTVAL INTO <variable_name> FROM <Table_name> or DUAL

  OR

  SELECT <sequence_name>.CURRVAL INTO <variable_name> FROM <Table_name> or DUAL

Executable Statements In PL/SQL

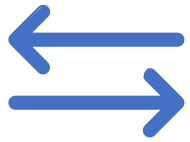
- In PL/SQL block you can use SELECT, INSERT, UPDATE and DELETE statements to manipulate data from the table
- DDL statements like CREATE, ALTER, DROP cannot be executed in PL/SQL
- DCL statements like GRANT and REVOKE are also not supported by PL/SQL
- SELECT statement can be used to retrieve data from the table using the following syntax
  - SELECT <select_list> INTO <variable_list> FROM <table_name> WHERE <condition>;
- The INTO clause is mandatory and occurs between the SELECT statement and FROM clause
- It is used to specify names of variables that holds the values that SQL returns from the SELECT clause
- You must specify one variable for each item selected and the order of the variables must correspond with the items selected. The SELECT query must return only one row
- The INSERT, UPDATE and DELETE statement will have the same syntax as they had in SQL with a minor change that they will be enclosed in BEGIN and END block

Nested Blocks

- One of the advantages of the PL/SQL is the ability to nest statement
- You can nest blocks whenever an executable statement is allowed thus making a nested block a statement
- The exception statement can also contain nested blocks
- The syntax for creating nested blocks can be
  - DECLARE
    - Variable Declaration
  - BEGIN
    - DECLARE
      - Variable Declaration
    - BEGIN
      - Executable Statements
    - END;
    - Executable Statements Of Outer Block
  - END;

PL/SQL Transactions

- Series of one or more SQL statements that are logically related or a series of operations performed on Oracle table data is termed as Transaction.
- Oracle treats this logical unit as a single entity and changes made to the table is a two-step process.
  - The changes requested are done
  - To make the changes permanent a COMMIT statement has to be given
  - To undo or reverse the changes a ROLLBACK statement is given



Closing Transactions

- A transaction can be closed by using either a COMMIT or ROLLBACK statement.
- By using these statements table data can be changed or all the changes made to the table are undone
- A COMMIT ends the current transaction and makes permanent any changes made during the transaction. All transactional locks acquired on the tables are released.
- Syntax
  - COMMIT;

Closing Transactions Contd..

- ROLLBACK does exactly opposite of COMMIT. It ends the transaction but undoes any changes made during the transaction. All transaction locks acquired on the tables are released.
- Syntax
  - ROLLBACK [TO [SAVEPOINT] <Save Point Name>];
  - SAVEPOINT is optional and is used to rollback a transaction partially, as far as the specified save point
  - Save Point Name is a save point created during the current transaction

Closing Transactions Contd..

- SAVEPOINT marks and saves the current point in the processing of a transaction.
- When a SAVEPOINT is used with ROLLBACK statement, parts of a transaction can be undone. An active SAVEPOINT is one that is specified since the last COMMIT or ROLLBACK
- SYNTAX
  - SAVEPOINT <Save Point Name>;

Closing Transactions Contd..

- ROLLBACK can be fired from the SQL prompt with or without the SAVEPOINT clause.
- A ROLLBACK operation performed without the SAVEPOINT clause amounts to the following
  - Ends the transaction
  - Undoes all the changes in the current transaction
  - Erases all the savepoints in that transaction
  - Releases all the transaction locks
- A ROLLBACK operation performed with the TO SAVEPOINT clause amounts to the following
  - A predetermined portion of transaction is rolled back
  - Retains the save point rolled back to but loses those created after the named savepoint
  - Releases all transactional that were acquired since the savepoint was taken

IF Statements

- The structure of the IF statement is similar to the structure of IF statement in other languages
- It allows PL/SQL to perform actions selectively based on conditions. The syntax of IF statement is

```
IF <condition> THEN
        statements;
[ELSIF <condition> THEN
        statements;]
[ELSE
        statements;]
END IF;
```

- condition is the Boolean expression or variable that returns TRUE, FALSE or NULL
- THEN introduces a clause that associates the Boolean expression with the sequence of statements that follows it
- Statements can be one or more PL/SQL or SQL statements. The statements in the THEN clause are executed only if the condition in the associated IF clause evaluates to TRUE

IF Statements Contd..

- ELSIF Is a keyword that introduces a Boolean expression
- ELSE Introduces a default clause that is executed if and only if none of the earlier conditions are TRUE
- END IF Marks the end of the IF statement
- ELSIF and ELSE are optional in an IF statement.
- You can have any number of ELSIF keywords but only one ELSE keyword in IF statement
- END IF marks the end of an IF statement and must be terminated by semicolon

IF Example – Find the smallest number

Direct/Static Input

--

```
declare
a int:=5;
b int:=6;
begin
if a < b then
DBMS_OUTPUT.PUT_LINE(a);
end if;
end;
```

User Input

--

```
Set serveroutput on
declare
a int;
b int;
begin
a:=&a;
b:=&b;
if a < b then
DBMS_OUTPUT.PUT_LINE(a);
else
DBMS_OUTPUT.PUT_LINE(b);
end if;
end;
```

IF ELSE Example- Find the number is even or odd

```
Set serveroutput on
DECLARE
   NO NUMBER;
BEGIN
   NO:=&no;
   IF MOD(NO,2) = 0 THEN
      DBMS_OUTPUT.PUT_LINE('EVEN NUMBER');
   ELSE
      DBMS_OUTPUT.PUT_LINE('ODD NUMBER');
   END IF;
END;
```

IF-ELSIF-ELSE Check number is positive or negative

```
Set serveroutput on
DECLARE
   NO NUMBER;
BEGIN
   NO:=&NO;
   IF NO < 0 THEN
      DBMS_OUTPUT.PUT_LINE('NEGATIVE NUMBER');
   ELSIF NO > 0 THEN
      DBMS_OUTPUT.PUT_LINE('POSITIVE  NUMBER');
   ELSE
      DBMS_OUTPUT.PUT_LINE('EQUAL TO ZERO');
   END IF;
END;
```

IF ELSE EXAMPLE –Find the grade of Stude..t


```
--
Set serveroutput on
declare
per number;
begin
per:=&per;
if per>=75 then
DBMS_OUTPUT.PUT_LINE('Your grade: A');
elsif per>=60 then
DBMS_OUTPUT.PUT_LINE('Your grade: B');
elsif per>=50 then
DBMS_OUTPUT.PUT_LINE('Your grade: C');
elsif per>=40 then
DBMS_OUTPUT.PUT_LINE('Your grade: D');
else
DBMS_OUTPUT.PUT_LINE('Your grade: F');
end if;
end;
```

CASE Expressions

- A CASE expression returns a result based on one or more alternatives.
- To return the result, the CASE expression uses a selector which is an expression whose value is used to return one of the several alternatives
- The selector is followed by one or more WHEN clauses that are checked sequentially
- The value of the selector determines which result is returned
- If the value of the selector equals the value of a WHEN clause expression that WHEN clause is executed, and the result is returned. The syntax of CASE expression is

```
CASE selector
        WHEN expression1 THEN result1
        WHEN expression2 THEN result2
        ....
        WHEN expressionN THEN resultN
        [ELSE resultN+1]
END;
```

Searched Case Expression

- PL/SQL also provides a searched CASE expression which has the following form

```
CASE
        WHEN search_condition1 THEN result1
        WHEN search_condition2 THEN result2
        ....
        WHEN search_conditionN THEN resultN
        [ELSE resultN+1]
END;
```

- A searched case expression has no selector
- Its WHEN clause contains search conditions that yield a Boolean value rather than expression that can yield a value of any type
- In searched CASE statements you do not have to test expression . Instead the WHEN clause contains an expression that results in a Boolean value

Case Expression example

```
--
Set serveroutput on
declare
per number;
Begin
per:=&per;
case per
when 8 then DBMS_OUTPUT.PUT_LINE('Your grade: A');
when 7 then DBMS_OUTPUT.PUT_LINE('Your grade: B');
when 6 then DBMS_OUTPUT.PUT_LINE('Your grade: C');
when 5 then DBMS_OUTPUT.PUT_LINE('Your grade: D');
else DBMS_OUTPUT.PUT_LINE('Your grade: F');
end case;
end;
```

Iterative Control : Loop Statements

- Loops are mainly used to execute statements or a sequence of instructions multiple times repeatedly until an exit condition occurs
- It is mandatory to have an exit condition in a loop else it becomes infinite
- Looping constructs are the second type of control structure
- PL/SQL provides the following types of loops
  - Basic loop that performs repetitive actions without overall conditions
  - FOR loops that perform iterative actions based on a count
  - WHILE loops that perform iterative action based on a condition

Iterative control

Simple loop

FOR Loop

The WHILE loop

Create a simple loop such that a message is displayed when a loop exceeds a particular value.

--

```
Set serveroutput on
declare
i number:=0;
begin
Loop
i:=i+2;
exit when i>10;
end loop;
DBMS_OUTPUT.PUT_LINE('the value of i has reached '|| (i));
End;
```

Basic Loop Examples

Print number between 1 to 10

```
--
DECLARE
        CTR NUMBER:=1;
BEGIN
        LOOP

        DBMS_OUTPUT.PUT_LINE(CTR);
                CTR:=CTR+1;
                EXIT WHEN CTR>10;
        END LOOP;
END;
```

Print factorial of a number using Loop.

```
--
DECLARE
        FACT NUMBER:=1;
        NO NUMBER:=&NO;
BEGIN
        LOOP
                FACT :=FACT * NO;
                NO:=NO-1;
                EXIT WHEN NO<1;
        END LOOP;
DBMS_OUTPUT.PUT_LINE(FACT);
END;
```

While Loops

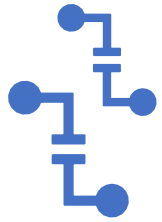
- WHILE loop can be used to repeat a sequence of statements until the controlling condition is TRUE.
- The condition is evaluated at the start of each iteration and if it evaluates to FALSE the loop terminates
- If the condition is FALSE at the beginning of the loop the loop may not execute at all
- The syntax for the WHILE loop is

```
WHILE <condition>
LOOP
        statements
END LOOP;
```

While Loop Example

- Display 1 to 10 using while loop

```
--
declare
a int:= 0;
begin
while a < 10
loop
a := a + 1;
DBMS_OUTPUT.PUT_LINE( a);
end loop;
end;
/
```



FOR Loops

- FOR loops have a control statement before the loop keyword to set the number of iterations that PL/SQL performs
- It has the same structure as that of the basic loop
- The syntax of the FOR loop is

```
FOR <variable> IN [REVERSE] lower_bound..upper_bound
LOOP
        statements;
END LOOP;
```

- In the above syntax the variable for the loop need not be declared as it is declared implicitly as integer
- The lower bound must be always less than upper bound
- REVERSE causes the loop to decrement with each iteration from the upper bound to lower bound

For Loop Examples

- Display 1 to 10 using for loop

```
--
begin
for i in 1..10
loop
dbms_output.put_line (i);
end loop;
end;
/
```

- Display 1 to 10 in descending order using for loop

```
--
begin
for i in reverse 1..10
loop
dbms_output.put_line (i);
end loop;
end;
/
```

SQL Cursor And Its Attributes

- Oracle server allocates a private memory area called context area for processing SQL statements. User have no control over this private area because it is totally controlled by Oracle
- A cursor is a pointer to this context area and it is automatically managed by Oracle server and hence it is called as Implicit cursor. When an executable block issues a SQL statement PL/SQL creates an implicit cursor. This cursor is named as SQL
- When a cursor is created by user it is called as Explicit cursor. It is used when a select statement fetches more than one row from a table and work with one row at a time
- Using the following SQL cursor attributes you can test the outcome of the SQL statements

SQL%FOUND	Boolean attribute that evaluates to TRUE if the most recent SQL statement returned at least one row
SQL%NOTFOUND	Boolean attribute that evaluate to TRUE if the most recent SQL statement did not return even one row
SQL%ROWCOUNT	An integer value that represents the number of rows affected by the most recent SQL statement

Create Table EMP

- create table emp (eid int, ename char(20),esal int);
- insert into emp values(1,'raj',40000);
- insert into emp values(2,'ravi',50000);
- insert into emp values(3,'priya',30000);
- insert into emp values(4,'neha',60000);
- select * from emp;
- EID ENAME ESAL
  1 raj 40000
  2 ravi 50000
  3 priya 30000
  4 neha 60000

State the implementation of %FOUND and %NOTFOUND attributes.

```
Set Serveroutput on
begin
   update  emp   set esal=&esal where eid=&eid ;
   if SQL%FOUND then

   dbms_output.put_line('record successfully updated');

   elsif SQL%NOTFOUND then
   dbms_output.put_line('employee number does not exist');

   end if;
   end;
```

State the implementation of %ROWCOUNT attribute.

```
Set serveroutput on
declare
    x number(5);
    begin
    update  emp   set  esal = esal + 1000;
    if SQL%FOUND then
    x := SQL%ROWCOUNT;
    dbms_output.put_line('Salaries for ' || x || 'employees are updated');
    elsif SQL%NOTFOUND then
    dbms_output.put_line('None of the salaries where updated');
    end if;
    end;
    /
```

Explicit Cursors

- A cursor that is opened for processing data through a PL/SQL block on demand is called as Explicit Cursor
- This cursor is declared and mapped to a SQL query in the Declare Section of the PL/SQL block and is used within an executable section
- An explicit cursor has the following attributes

Attribute Name	Description
%ISOPEN	Returns TRUE if cursor is open else FALSE
%FOUND	Returns TRUE if record is fetched successfully else FALSE
%NOTFOUND	Returns TRUE if record is not fetched successfully else FALSE
%ROWCOUNT	Returns the number of records processed from the cursor

Create Table EMP

- create table emp (eid int, ename char(20),esal int)
- insert into emp values(1,'raj',40000)
- insert into emp values(2,'ravi',50000)
- insert into emp values(3,'priya',30000)
- insert into emp values(4,'neha',60000)
- select * from emp;
- EID ENAME ESAL
  1 raj 40000
  2 ravi 50000
  3 priya 30000
  4 neha 60000

Explicit Cursor Examples

- create table emp1(eid int, ename char(20),esal int);
- Cursor Implementation:

```
declare
cursor c1 is select * from emp;
x emp%rowtype;
begin
open c1;
fetch c1 into x;
insert into emp1 values(x.eid, x.ename, x.esal);
close c1;
end;
/
```

Explicit Cursor Examples

```
create table emp2(eid int, ename char(20),esal int)
declare
cursor c1 is select * from emp;
x emp%rowtype;
begin
open c1;
for i in 1..4
loop
fetch c1 into x;
insert into emp2 values(x.eid, x.ename, x.esal);
end loop;
close c1;
end;
```

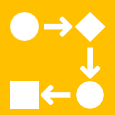
```
create table emp3(eid int, ename char(20),esal int)

declare
cursor c1 is select * from emp;
begin
for x in c1
loop
insert into emp3 values(x.eid, x.ename, x.esal);
end loop;
end;
```

Procedures / Functions

A Procedure or Function is a logically grouped set of SQL and PL/SQL statements that perform a specific task.

A stored procedure or function is named PL/SQL code that has been compiled and stored in Oracle



Procedures or Functions can be made dynamic by-passing parameters to it.

A Procedure or Function can then change the way it works depending upon the parameters passed

Procedures

- The syntax of procedure is

CREATE OR REPLACE PROCEDURE <Procedure Name>(<Argument> {IN, OUT, IN OUT} <Data Type>,.....) {IS, AS}

<Variable Declarations>;

<Constants Declaration>;

BEGIN

<PL/SQL sub program body>;

EXCEPTION

<Exception PL/SQL block>;

END; where

- Procedure Name is the name of the procedure to be created
- Argument is the name of the argument to be passed to the procedure
- IN (Default) indicates the parameter will only accept a value from the user
- OUT indicates the parameter will only return a value to the user
- IN OUT indicates the parameter will accept as well as return a value to and from the user
- Data Type indicates the data type of the associated parameter

Functions

- The syntax of function is

CREATE OR REPLACE FUNCTION <Function Name>(<Argument> IN <Data Type>,....) RETURN <Data Type> {IS, AS}

<Variable Declarations>;

<Constants Declaration>;

BEGIN

<PL/SQL sub program body>;

EXCEPTION

<Exception PL/SQL block>;

END; where

- Function Name is the name of the function to be created
- Argument is the name of the argument to be passed to the procedure
- IN indicates the parameter will only accept a value from the user
- Data Type indicates the data type of the associated parameter
- RETURN Data Type (Compulsory) indicates the data type of the functions return value

Procedures / Functions

- Invoking a Procedure can be done as follows

  Procedure Name(Argument List);

  OR

  CALL Procedure Name(Argument List);

- Invoking a Function can be done as follows

  Return Variable := Function Name(Argument List);

- Deleting a stored procedure

  DROP PROCEDURE <Procedure Name>;

- Deleting a stored function

  DROP FUNCTION <Function Name>;

Example For Procedure

- Write a procedure to add two numbers and call that procedure through the PL/SQL block

Syntax:

```
CREATE OR REPLACE PROCEDURE ADD_TWO_NOS
   IS
   A NUMBER(3):=10; B NUMBER(3):=20; C NUMBER(5);
BEGIN
   C:=A+B;
   DBMS_OUTPUT.PUT_LINE('SUM OF ' || A || ' AND ' || B || ' IS ' || C);
END;
/
```

Calling procedure:

```
BEGIN
   ADD_TWO_NOS;
END;
/
```

- sql>ed d:\f1.sql
- CREATE OR REPLACE PROCEDURE ADD_TWO_NOS IS
- A NUMBER(3):=10;
- B NUMBER(3):=20;
- C NUMBER(5);
- BEGIN
- C:=A+B;
- DBMS_OUTPUT.PUT_LINE('SUM OF ' || A || ' AND ' || B || ' IS ' || C);
- END;
- /
- To run type:
- sql>@ d:\f1.sql
- o/p: procedure created
- sql>ed d:\f2.sql
- BEGIN
- ADD_TWO_NOS;
- END;
- /
- sql>@ d:\f2.sql
- o/p: SUM OF 10 AND 20 IS 30
- PL/SQL procedure successfully completed.

Example For Procedure

- Write a procedure to add two numbers passed from the calling PL/SQL block.

```
CREATE OR REPLACE PROCEDURE ADD_TWO_PARAMETERNOS(A IN NUMBER,B IN NUMBER)
   IS
   C NUMBER(5);
BEGIN
   C:=A+B;
   DBMS_OUTPUT.PUT_LINE('SUM OF ' || A || ' AND ' || B || ' IS ' || C);
END;
/
DECLARE
   NUM_1 NUMBER(3); NUM_2 NUMBER(3);
BEGIN
   NUM_1:=&NUM_1; NUM_2:=&NUM_2;
   ADD_TWO_PARAMETERNOS(NUM_1,NUM_2);
END;
/
```

Example For Procedure

- Write a procedure to add two numbers passed from the calling PL/SQL block. The addition is done in the procedure but display of the sum is done in the calling program.

```
CREATE OR REPLACE PROCEDURE ADD_TWO_PARAMETERNOS(A IN NUMBER,B IN NUMBER,C OUT
   NUMBER) IS
BEGIN
   C:=A+B;
END;
/
DECLARE
   NUM_1 NUMBER(3); NUM_2 NUMBER(3); NUM_3 NUMBER(5);
BEGIN
   NUM_1:=&NUM_1; NUM_2:=&NUM_2;
   ADD_TWO_PARAMETERNOS(NUM_1,NUM_2,NUM_3);
   DBMS_OUTPUT.PUT_LINE('SUM OF ' || NUM_1 || ' AND ' || NUM_2 || ' IS ' || NUM_3);
END;
/
```

Example For Functions

- Write a function to add two numbers passed to it from main. The two numbers are accepted by them from the users

```
CREATE OR REPLACE FUNCTION ADD_TWO_NOS3(A IN NUMBER,B IN NUMBER) RETURN NUMBER IS
        C NUMBER(3);
BEGIN
        C:=A+B;  RETURN C;
END;
/
DECLARE
        X NUMBER(3); Y NUMBER(3); Z NUMBER(3);
    BEGIN
        X:=&X;    Y:=&Y;
        Z:=ADD_TWO_NOS3(X,Y);
        DBMS_OUTPUT.PUT_LINE('VALUE OF Z IS  '||Z);
END;
/
```

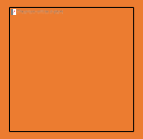
Write a PL/SQL function which will compute and return the maximum of two values:


```
Create FUNCTION findMax(x IN number, y IN number)  RETURN number  IS      z number;
BEGIN
   IF x > y THEN     z:= x;  ELSE     z:= y;
   END IF;
   RETURN z;
END;
```

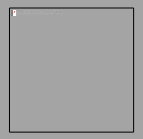
Calling function

```
DECLARE
   a number;   b number;    c number;
BEGIN
   a:=&a;      b:= &b;
   c := findMax(a, b);
   dbms_output.put_line(' Maximum of (a, b): ' || c);
END;
```

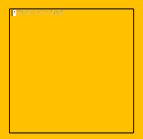
Packages



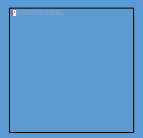
Is an Oracle object which holds other objects like procedures, functions, variables, constants, cursors and exceptions.



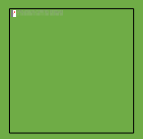
It is a way of creating generic encapsulated reusable code



A package once written and debugged is compiled and stored in Oracles systems table held in Oracle database



Packages can contain PL/SQL blocks of code which have been written to perform some process entirely on their own.



These PL/SQL blocks of code called subprograms as do not require any kind of input from PL/SQL blocks of code

Advantages Of Packages

- Packages enable the organization of commercial applications into efficient modules. Each package is easily understood and the interfaces between the packages are simple clear and well defined.
- Packages allow granting of privileges efficiently.
- A packages public variables and cursors persists for the duration of the session. Therefore all cursors and procedure that execute in this environment can share them
- Packages enable overloading of procedures and functions when required
- Packages improve performance by loading multiple objects into memory at once. Therefore subsequent calls to related subprograms in the package requires no I/O
- Packages promote code reuse through the use of libraries that contain stored procedures and function thereby reducing redundant coding

Components Of Oracle Package

- A package has two components
  - Package specification which declares the types, memory variables, constants etc. The package specification contains name of package, names of data types of any arguments. This declaration is local to the database and global to the package
  - Package body which fully defines cursors, functions and procedures thus implementing the specification. The body of the package contains the definition of public objects that are declared in the specification. The body can also contain other object declarations that are private to the package. The objects declared privately in the package body are not accessible to the other objects outside the package. The package body can contain subprogram bodies

Syntax-Creating A Package Specification

- Package Specification

```
CREATE OR REPLACE PACKAGE <package_name> IS/AS
        FUNCTION <function_name> (<list of arguments>)
        RETURN <datatype>;
        PROCEDURE <procedure_name> (<list of arguments>);
        -- code statements
END <package_name>;
```

Package Body

Syntax-Creating A Package Body

```
CREATE  OR REPLACE PACKAGE BODY <package_name> IS/AS
FUNCTION <function_name> (<list of arguments>) RETURN <datatype>IS/AS
        -- local variable declaration;
        BEGIN
                -- executable  statements;
        EXCEPTION
                -- error handling statements;
END <function_name>;

PROCEDURE <procedure_name> (<list of arguments>)IS/AS
        -- local variable declaration;
        BEGIN
                -- executable statements;
        EXCEPTION
                -- error handling statements;
        END <procedure_name>;
END <package_name>;
```

Invoking Packages SubPrograms

- To reference a packages subprograms and objects use dot notation as given below
- PackageName.Type_Name;
- PackageName.Object_Name;
- PackageName.Subprogram_Name;

Where

- PackageName is the name of the declared package
- Type_Name is the name of the type that is user defined such as record
- Object_Name is the name of the constant or variable that is declared by the user
- SubProgram_Name is the name of the procedure or function contained in the package body

```
BEGIN
   EMPACTIONS.NEWEMPLOYEE(---Values to entered in the table);
END;
```

Example – Area of a Circle with PLSQL Package

- Create or replace package area_shapes as
- function area(radius number)
- return number;
- end area_shapes;
- /

- Output:
- Package created.

Example – Area of a Circle with PLSQL Package

- Create or replace package body area_shapes as
- function area(radius number)
- return number
- is
- area1 number;
- begin
- area1 := 3.142*radius*radius;
- return area1;
- end;
- end area_shapes;
- /
- Output
- Package body created.

Final Output: Calling of function

```
SQL> select area_shapes.area(2)  from dual;

AREA_SHAPES.AREA(2)
-------------------
             12.568
```

Composite Data Types

- A variable of composite data type can hold multiple values of scalar data type or composite data type
- Composite data types are
  - PL/SQL records – Used to treat related but dissimilar data as a logical unit. A PL/SQL record can have variables of different types. For example you can define a record to hold employee details
  - PL/SQL collections – Used to treat data as a single unit. Collections are of three types INDEX BY TABLES or associative arrays, Nested table, VARRAY

PL/SQL Records

- A record is a group of selected data items stored in fields each with its own name and data type
- Each record defined can have as many fields as necessary
- Records can be assigned initial values and can be defined as NOT NULL
- Fields without initial values can be initialized to NULL
- The DEFAULT keyword can also be used when defining fields.
- You can define RECORD types and declare user defined records in the declarative part of PL/SQL block
- The syntax for creating PL/SQL record is

```
TYPE <Type Name> IS RECORD
(field name_1 field type,
field name_2 field type, ....
);
```

Example

- declare
- type abc is record
- (
- x int,
- y char(10));
- p abc;
- begin
- select eid, ename into p from e2 where esal=40000;
- insert into e1 values(p.x, p.y);
- end;

ERROR/EXCEPTION HANDLING

- While executing SQL sentences anything can go wrong and the SQL sentence can fail.
- When SQL sentence fails the Oracle engine is the first to recognize this as an Exception Condition.
- It immediately tries to handle the exception condition and resolve it by raising a built-in Exception Handler.
- An Exception Handler is nothing but a code block in memory that will attempt to resolve the current exception condition.
- To handle very common and repetitive conditions the Oracle engine uses Named Exception Handlers.
- It has about 15 to 20 named exception handlers and 20,000 numbered exception handlers.
- Each exception handler has a code attached that will attempt to resolve an exception condition.
- This process is called as DEFAULT EXCEPTION HANDLING STRATEGY
- Oracles default exception handling code can be overridden by which the code given in the exception section is executed which is called as USER DEFINED EXCEPTION HANDLING CODE

ERROR/EXCEPTION HANDLING Contd

- When default handling code is overridden by the code given in the EXCEPTION section of the PL/SQL block it means that it is an example of a programmer giving explicit exception handling instructions to an Oracle exception handler
- This means that the Oracle engine's exception handler must decide whether it has to execute its own exception handling code or whether it has to execute user defined exception handling code.
- As soon as the Oracle engine invokes an exception handler the exception handler goes back to the PL/SQL block from which the exception condition was raised.
- It scans the PL/SQL block for the existence of an EXCEPTION section within the PL/SQL block.
- If the EXCEPTION section exists the exception handler scans the first word after WHEN in this exception section and if the first word is exception handlers name then it executes the code given after the THEN keyword
- The syntax for the EXCEPTION section is
    - EXCEPTION
        - WHEN <Exception Name> THEN
            - <User Defined Action To Be Carried Out>

ERROR/EXCEPTION HANDLING EXAMPLE

```
DECLARE
   A NUMBER(3);
   B NUMBER(3);
   C NUMBER(3);
BEGIN
   A:=&A;
   B:=&B;
   C:=A/B;
   DBMS_OUTPUT.PUT_LINE(' VALUE OF C IS ' || C);
EXCEPTION
   WHEN ZERO_DIVIDE THEN
            DBMS_OUTPUT.PUT_LINE('DIVIDE BY ZERO IS NOT POSSIBLE');
END;
/
```

This example shows how the in built exception ZERO_DIVIDE can be used in the PL/SQL block

The VALUE_ERROR Exception

```
declare
x char(4);
begin
x:='Too big';
insert into temp values(1,x);
exception when value_error then
insert into temp values(0,'value
error');
end;
/
```

```
declare
x char(4);
begin
x:='Too';
insert into temp values(1,x);
exception when value_error then
insert into temp values(0,'value error');
end;
/
```

- Oracle's Named Exception Handlers
- Oracle has a predefined error handlers called as Named Exceptions and are referred by their names.

ZERO_DIVIDE	Raised when a number is divided by zero
NO_DATA_FOUND	Raised when a select statement returns zero rows
INVALID_NUMBER	Raised when a program fails to convert a string into a number
DUP_VAL_ON_INDEX	Raised when an insert or update attempts to create two rows with duplicate values in columns constrained by unique index
LOGIN_DENIED	Raised when an invalid username or password was used to log onto Oracle
NOT_LOGGED_ON	Raised when Pl/SQL issues an Oracle call without being logged on to Oracle
PROGRAM_ERROR	Raised when PL/SQL has an internal problem
TIMEOUT_ON_RESOURCE	Raised when Oracle has been waiting to access a resource beyond the user defined time out limit
TOO_MANY_ROWS	Raised when a select statement returns more than one row
VALUE_ERROR	Raised when the data type or data size is invalid
OTHERS	Stands for all other exceptions not explicitly named

User Named Exception Handlers

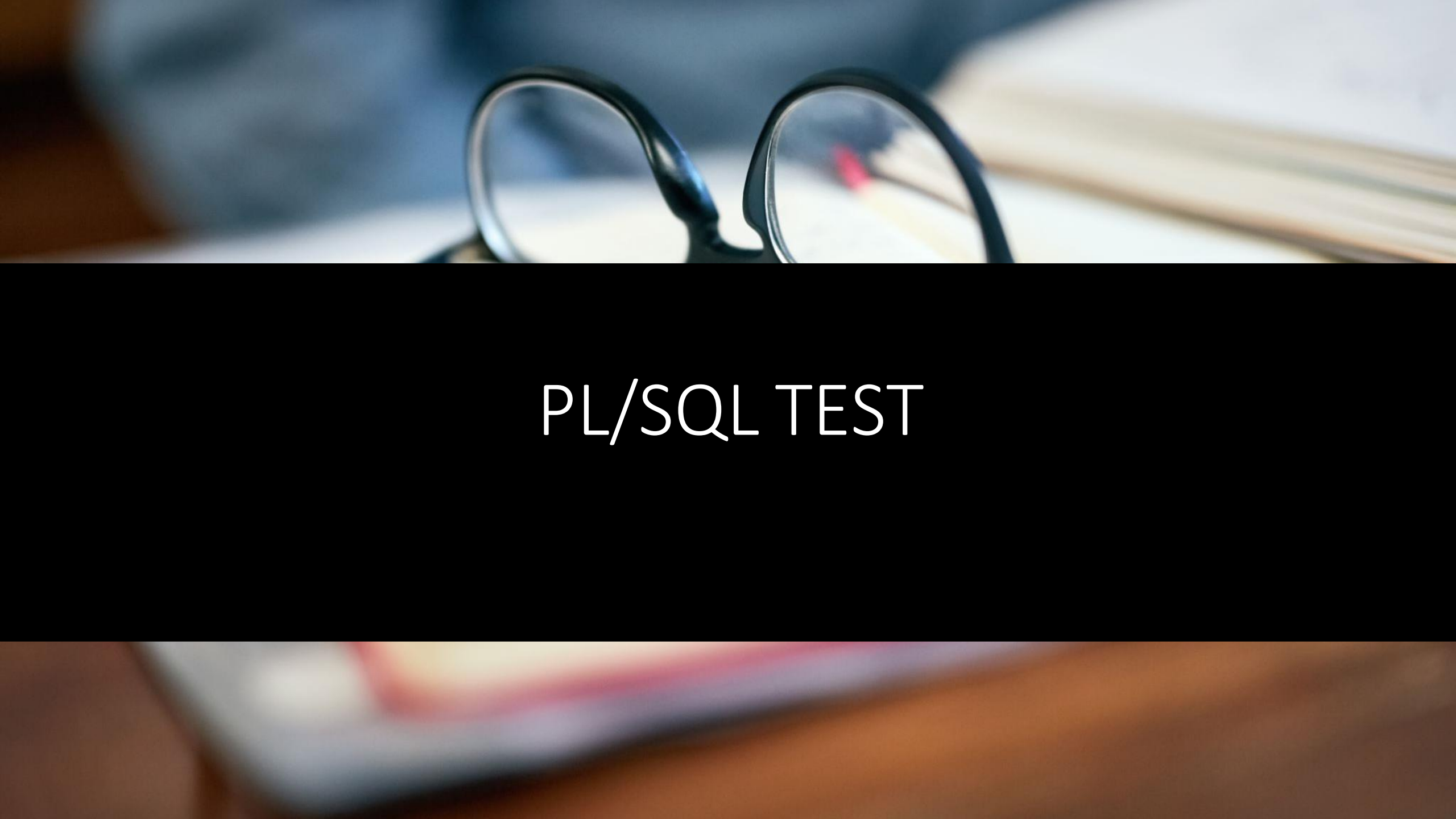
- In commercial applications data being manipulated needs to be validated against business rules. If the data violates a business rule the entire record must be rejected.
- To trap business rules being violated the technique of raising user defined exceptions and then handling them is used. User defined error conditions must be declared in the declarative part of PL/SQL block.
- In the executable part a check for the condition that needs special attention is made and if the condition exists the call to the user defined exception is made using RAISE statement. The exception once raised is then handled in the Exception handling section of the PL/SQL code block .
- The syntax for declaring the exception is

```
DECLARE
    Exception Name EXCEPTION;
BEGIN
    <SQL statement>;
    IF <Condition> THEN
            RAISE <Exception Name>;          END IF;
EXCEPTION
    WHEN <Exception Name> THEN
            <User Defined Action To Be Taken>; END;
```

User Defined Exception Example

- Write a PL/SQL block to calculate the area of a rectangle. The user enters the length and the breadth of the rectangle. If any of these two values entered by the user are negative or zero an exception should be raised giving an appropriate message.

```
DECLARE
   L NUMBER(3); B NUMBER(3); AREA NUMBER(6);
   VALID_SIDE EXCEPTION;
BEGIN
   L:=&L;   B:=&B;
   IF L<1 OR B<1 THEN
            RAISE VALID_SIDE;
   END IF;
   AREA:=L*B;
   DBMS_OUTPUT.PUT_LINE('AREA OF RECTANGLE IS '||AREA);
EXCEPTION
   WHEN VALID_SIDE THEN
            DBMS_OUTPUT.PUT_LINE('EITHER LENGTH OR BREADTH IS OF INVALID VALUE');
   WHEN OTHERS THEN
            DBMS_OUTPUT.PUT_LINE('ERROR DUE TO SOME INTERNAL PROBLEM'); END;
```



PL/SQL TEST

Thank You